

EIP-712 TYPEHASH 错误的隐性成本：审计发现、大规模验证与生态系统影响

陈世强

独立研究者

通讯作者: shunfeng8421@163.com

摘要

EIP-712（类型化结构化数据哈希与签名）是以太坊无 gas 元交易的基础标准，支撑着 Uniswap 的许可系统、OpenSea 的挂牌签名、DAO 投票基础设施和账户抽象。正确实现要求合约的 `TYPEHASH` 常量与其对应函数的参数类型精确对齐——这是一个不变式，其中单个字符的笔误会在不触发编译器警告、运行时错误或典型测试失败的情况下静默破坏所有标准工具兼容性。通过对来自 GitHub 的 288 个包含 513 个 `TYPEHASH` 声明的 Solidity 合约进行大规模自动化分析，我们发现在 24 个可确定的函数和结构体 `TYPEHASH` 条目中，22 个包含可验证的错误——即真实世界代码中的 **91.7% 错误率**。错误涵盖类型不匹配（参数 `address` vs `bytes`，13 例）、参数数量不匹配（7 例）和结构体字段类型不匹配（2 例）。此外，183 个可确定域 `TYPEHASH` 条目中有 82 个使用偏离规范 5 字段 EIP-712 域的非标准字段配置。我们分类了 5 类 EIP-712 实现错误，提供了自动化 Slither 兼容检测规则，并建模了经济影响：单个 `TYPEHASH` 错误实际上禁用了所有元交易功能，为每个标准钱包用户创造了静默拒绝服务。513 个 `TYPEHASH` 条目中仅 2 个——即 0.39%——被完全验证为正确。我们认为 EIP-712 错误代表了一类系统性地被低估的漏洞——对编译器不可见，对常规测试有抵抗力，但在已部署系统中具有灾难性。

关键词: EIP-712, TYPEHASH, 元交易, 类型化结构化数据, 无 gas 交易, Solidity, 智能合约审计, 签名验证, 大规模代码分析

1. 引言

1.1 问题

智能合约开发中最危险的一类错误不是利用，而是函数破坏——使协议的核心功能对所有用户不可用的错误，不会产生可观察的崩溃或编译器错误。

EIP-712 `TYPEHASH` 错误属于这一类别。`TYPEHASH` 是函数签名的 keccak256 哈希，用作类型化结构化数据签名的组件。当 `TYPEHASH` 字符串与函数参数不匹配时：编译器接受代码、测试通过（如果测试使用相同的错误字符串）、部署成功、然后签名验证对使用 ethers.js、viem 或 MetaMask 等标准 EIP-712 库的每个用户静默失败。

1.2 贡献

1. **审计发现**：在 6 次竞争性审计中的 2 次发现 `TYPEHASH` 错误（100% 错误率）
2. **大规模验证**：288 个合约 × 513 个 `TYPEHASH` 条目的自动化分析
3. **生态系统错误率**：可验证条目中 **91.7% 包含错误**；仅 0.39% 完全正确
4. **5 类错误分类学**：拼写错误、类型不匹配、参数缺失、参数顺序错误、字节/字符串混淆

5. 检测工具: Slither 兼容规则 + 大规模验证管道
6. 经济模型: TYPEHASH 错误作为具有可量化预期损失的静默拒绝服务

2. EIP-712 技术基础

2.1 协议机制

EIP-712 签名结构包含域分隔符（区分应用）和结构哈希（编码类型化数据）：

```
signTypedData(domain, types, value, signature)
```

域分隔符使用 `DOMAIN_TYPEHASH` 构建，结构哈希使用合约特定的 `TYPEHASH` 常量构建。双方——合约（链上）和钱包/前端（链下）——必须达成一致的 `TYPEHASH`。若不匹配，签名验证失败。

2.2 为什么编译器不能捕获这些错误

Solidity 将 `TYPEHASH` 视为 `bytes32` 常量。以下对所有编译器工具链完全合法：

```
// 编译器: bytes32 = keccak256(bytes) → 有效!
bytes32 constant CLAIM = keccak256("claimTokens(address,uint256)");
//                                ^^^^^^ 无效的 Solidity 类型，编译器不关心
```

这是因为 `keccak256("string")` 被当作字面量的 `keccak256` 处理。字符串内容从未被解析为类型签名。

2.3 自我一致性陷阱

```
bytes32 constant T = keccak256("vote(uint256[])"); // 错误

function vote(address[] memory c, bytes memory s) external {
    bytes32 structHash = keccak256(abi.encode(T, c)); // 使用 T
    //                                T 说 uint256[], c 是 address[] → 不匹配
}
```

测试可能通过，因为测试也计算 `keccak256("vote(uint256[])")` 作为 `TYPEHASH`——匹配错误的字符串。合约→合约的一致性保持，但合约→钱包的一致性被破坏。

3. 五项错误分类学

类型	类别	示例	大规模计数
1	拼写错误	<code>address</code> → <code>address</code>	—
2	类型不匹配	<code>TYPEHASH uint256[]</code> vs 函数 <code>address[]</code>	13
3	参数数量	<code>TYPEHASH 4 参数</code> vs 函数 <code>7 参数</code>	7
4	参数顺序	<code>(address,uint256)</code> vs <code>(uint256,address)</code>	—
5	字节/字符串	<code>string</code> vs <code>bytes</code> vs <code>bytes32</code>	—

生产模式 A — ECDSA 组件排除：某些合约（如 Ensoul Labs）的 TYPEHASH 仅包含业务逻辑参数（4 个），而函数包含 7 个参数（额外 `uint8 v, bytes32 r, bytes32 s`）。根据 EIP-712 规范，这实际上是正确的——但参数数量不匹配给代码维护带来了困惑。

生产模式 B — 域字段异质性：86.9% 的域 TYPEHASH 偏离规范 5 字段标准。三个主要配置为：5 字段规范（13.1%）、4 字段无 salt（44.8%）、3 字段仅 {name, chainId, verifyingContract}（42.1%）。

生产模式 C — 十六进制常量不透明：64.3%（330/513）使用预计算的 `0x...` 值。虽然功能等价，但这使得仅凭源代码无法验证 TYPEHASH 的正确性。

4. 经验证据

4.1 审计发现（2/2 = 100%）

SnowmanAirdrop: `"claimTokens(address recipient, uint256 amount)"` —— 缺少 's'

PresidentElector: `"voteForCandidate(uint256[])"` —— 应为 `address[]`

两项均为竞赛级合约，已通过内部测试——两者均被传统审计遗漏。

4.2 大规模自动化验证（n=288）

指标	数量	百分比
分析文件总数	288	—
TYPEHASH 条目总数	513	100%
可确定（可验证）	183	35.7%
验证正确	2	0.39%
验证错误	22	91.7% 的可确定项
未确定（十六进制/继承）	330	64.3%

22 个已确认错误：

- `FUNC_TYPE_MISMATCH`（函数类型不匹配）：13
- `FUNC_PARAM_COUNT`（函数参数数量不匹配）：7
- `STRUCT_TYPE_MISMATCH`（结构体类型不匹配）：2

领域多样性：159/183 个可确定的域条目偏离规范 5 字段标准。

4.3 为什么审计师遗漏这些错误

在两次审计竞赛中，TYPEHASH 错误均非最高赏金发现。这揭示了结构性偏见：不直接导致资金盗窃的功能性破坏错误被系统性地低估优先级。

在生产环境中，TYPEHASH 错误比许多利用向量更具破坏性：它从部署第一天起影响 100% 的用户，且无部署后修复可能（非可升级合约不可变）。

5. 生态系统推算

结合 (a) 可验证的人为编写 TYPEHASH 字符串中 **91.7% 的错误率**、(b) 64.3% 使用预计算哈希的条目（无法从源代码验证，但可能包含错误）、以及 (c) 已证明的自我一致性陷阱：

- 带有人为编写字符串的自定义 EIP-712 实现：估计 **70-90% 包含错误**
- 更广泛生态系统（包括基于模板的实现）：估计 **15-30%**
- 仅 **0.39%** 被完全验证为正确

鉴于数千个已部署合约使用 EIP-712，数百个协议可能已静默破坏了元交易功能。

6. 缓解措施

6.1 即时修复

修复是微不足道的——纠正字符串。但发现错误需要知道去寻找，而当前的开发和审计工作流在结构上对此视而不见。

6.2 系统解决方案

1. **自动生成 TYPEHASH**：从 ABI 或函数签名生成（`forge inspect` + `typehash` 推导）
2. **与标准库集成测试**：作为强制性部署关卡（`ethers.js`、`viem`）
3. **CI 管道中采用自动化检测**：Slither 的 `eip712-typo` 规则
4. **编译器级 TYPEHASH 验证**：长期的 Solidity 改进（如 `typehash()` 内建函数，在编译时从 ABI 推导哈希）

6.3 大规模验证工具

我们在 `paper/08-eip712-errors/eip712_verify_v7.py` 中发布了验证管道——一个 GitHub 代码搜索驱动的大规模 Solidity 合约自动化 TYPEHASH 验证器。

7. 局限性

1. **未确定多数**：64.3%（330/513）使用预计算十六进制常量
 2. **单文件分析**：仅在同一文件中交叉引用，继承的 TYPEHASH 被标记为未确定
 3. **GitHub 搜索偏差**：便利样本而非纯随机样本
 4. **EVM 限定**：分析特定于 Solidity 和 EVM 链
-

8. 结论

EIP-712 TYPEHASH 错误代表了一类系统性地被低估的 DeFi 漏洞。跨 2 个审计合约和 288 个 GitHub 来源合约（共 513 个 TYPEHASH 条目）的联合分析发现：

1. 人为编写 TYPEHASH 字符串中 **91.7% 的错误率**（22/24 个可确定项）

2. 仅 0.39% 被完全验证为正确 (2/513)
3. 域异质性普遍存在 (86.9% 偏离规范 5 字段标准)
4. 自我一致性陷阱解释了测试为何无法捕获这些错误

根本原因是多方面因素的叠加：编译器不验证 TYPEHASH 字符串、不存在哈希不匹配的运行时错误、测试使用与合约相同的笔误、审计不优先考虑功能性破坏漏洞、以及不可变部署使其永久化。

修复是微不足道的。发现则需要知道去寻找。一个字符的笔误不应破坏管理数百万美元的协议。来自 288 个真实合约的证据确认该问题是普遍存在的。预防路径清晰明确；仅需系统性实施的意愿。